DFT: Discrete Fourier Transform & Linear Signal Processing

2nd Year Electronics Lab

IMPERIAL COLLEGE LONDON

Table of Contents

Equipment 2

Aims 2

Objectives 2

Recommended Textbooks 3

Recommended Timetable 3

1. Introduction 3

Exercise 1 – Learning MATLAB 3

Exercise 2 – Three forms of Fourier Transforms 4

Exercise 3 – DFT and Spectra of Signals 6

Exercise 4 – The effects of using Windows 8

Exercise 5 – The analysis of an unknown signal 8

2. What is a Digital Filter? 9

Exercise 6 – Filtering in the Frequency Domain 9

3. Impulse Response and Convolution 9

Exercise 7 – Spectra Pulse and Impulse 10

4. Frequency Response of a Filter 10

Exercise 8 – A simple Finite Impulse Response (FIR) Filter 10

Exercise 9 – Infinite Impulse Response (IIR) Filter 11

Equipment

- Lab Computer with **MATLAB**

Aims

This experiment leads you through the theory of **Discrete Fourier Transforms** and **Discrete Time Signal Processing**. These have many similarities to **Continuous Fourier Transforms**, covered in Year 1, but are not identical.

- Familiarization with **MATLAB**
- Able to generate discrete signals in **MATLAB**
- Perform analysis on signals by applying Fourier Transforms and Windows
- Perform analysis on digital filters and their responses

Objectives

- Derive equations for the 3 types of **Discrete Fourier Transforms**.
- Generate sinusoidal signals in **MATLAB** as vectors and investigate the effects of **DFT** and **Windowing**. You will then use these techniques to investigate an unknown signal (provided).
- Filter noise from the unknown signal by removing unwated frequencies in the frequency domain.
- Investigate the effects of passing **Pulse** and **Impulse** signals through a digital filter.
- Investigate simple digital filter and their responses (**FIR** and **IIR**).

Recommended Textbooks

This experiment is designed to support the second year course Signals and Linear Systems (EE2-5). Since the laboratory and the lectures may not be synchronized to each other, some of you may need to learn certain aspects of signal processing ahead of the lectures. While the notes provided in this experiment attempt to explain certain basic concepts, it is far from complete.

You are recommended to refer to the excellent textbook '**An Introduction to the Analysis and Processing of Signals**' by Paul A. Lynn. A more advanced reference is '**Discrete-Time Signal Processing**', by A.V. Oppenheim & R.W. Schafer, as well as the 'Signals and Linear Systems' course notes. Another good recommendation is '**Signals and Systems (2nd Edition)**' by A.V. Oppenheim, Alan S. Willsky with S. Hamid Nawab.

Recommended Timetable

There are 9 exercises in this handout; you should aim to complete all of them in the 4 timetabled lab sessions.

1. Introduction

Apart from teaching you some fundamentals of **Digital Signal Processing (DSP)**, this experiment also introduces you to **MATLAB**, a mathematical tool that integrates numerical analysis, matrix computation and graphics in an easy-to-use environment. **MATLAB** is highly interactive; its interpretative nature allows you to explore mathematical concepts in signal processing without tedious programming. Once grasped, the same tool can be used for other subjects such as circuit analysis, communications, and control engineering.

Exercise 1 – Learning MATLAB

Launch **MATLAB** from your computer.

NOTE: the online help is readily available within **MATLAB** by simply typing `help` at the cursor input '>'.

Let us create a sampled sine wave at a frequency f_{sig} of 1KHz. We shall be using a sampling frequency of 25.6KHz and generating 128 data points. How many cycles of the sine wave will there be?

To do this, a list of time points are needed to be created (0, $1/f_{samp}$, $2/f_{samp}$, ..., $127/f_{samp}$). In **MATLAB**, create a vector with 128 elements by entering the following:

```
% Define constants and the time vector t
fsamp= 25600
fsig = 1000
tsamp = 1/fsamp
t = 0 : tsamp : 127*tsamp;
```

MATLAB tips:

- You do not need to declare variables.
- Anything between `%` and newline is a comment.
- The result of the **MATLAB** statement is echoed unless the statement terminates with a semicolon ';'.
- The colon ':' has special significance. For example, `x=1:5` creates a row vector containing five elements, vector x=[1 2 3 4 5]. In this example it creates a vector `t` with the first element 0, the last element 127*t_{samp}, and with elements in between at an increment of t_{samp}.

Now try the command `whos` to check which variables are defined at this stage and also how large they are (matrices are reported as row by column).

To create the sine wave **y** and plot it against **t** simply enter:

```
y = sin(2*pi*fsig*t);
plot (y)
```

This plots **y** against the data number (from 1 to 128). Alternatively use **plot(t, y)** for plotting **y** against **t**. To add more thrills, try:

```
grid
title ('Simple Sine Wave')
xlabel ('Time')
ylabel ('Amplitude')
```

Obtain hardcopy of graphics with the **print** command. You may also plot the sine wave as points by using **plot(t, y, '*')** and change the colour of the curve to red with **plot(t, y, 'r')**. The output can be given as a discrete signal using the command **stem(y)**. The plot window can also be divided with the **subplot** command and the **zoom** command allows examination of a particular region of the plot (use help to discover more).

Sine waves are such useful waveforms that it is worthwhile to write a function to generate them for any signal frequency, sampling frequency and any number of samples. In **MATLAB**, functions are individually stored as a file with extension '**.m**'. To define a function **sinegen**, create a file **sinegen.m** using the **MATLAB** editor by selecting **New M-file** from the **File** menu.

```
function y = sinegen(fsamp, fsig, nsamp)
***** body of the function *****
end
```

Save the file in your home directory with the **Save As** option from the **File** menu. To test **sinegen**, simply enter:

```
s = sinegen(*****SUPPLY PARAMETERS*****);
```

If **MATLAB** gives the error message **undefined function**, use the **cd** command to make your home directory the local directory.

Exercise 2 – Three forms of Fourier Transforms

We have seen how **MATLAB** can manipulate data as a 1x128 vector. Now let us create a matrix **S** containing 4 rows of sine waves such that the first row is frequency f_{sig}, the second row is the second harmonic $2*f_{sig}$ and so on. **nsamp** should be set to equal 128.

```
S(1,:) = sinegen(fsamp, fsig, nsamp);
S(2,:) = sinegen(fsamp, 2*fsig, nsamp);
S(3,:) = sinegen(fsamp, 3*fsig, nsamp);
S(4,:) = sinegen(fsamp, 4*fsig, nsamp);
```

Note the use of '**:**' as the second index of **S** to represent all columns. Examine **S** and then try plotting it by entering **plot(S)**. Is this plot correct? If not, why? Plot the transpose of **S** by using **plot(S')**.

We can do the same thing to the above 4 statements by using a for-loop. The following statements creates **S** with 10 rows of sine waves:

```
for i = 1:10
S(i,:) = sinegen(fsamp, i*fsig, nsamp);
end
```

Next let us explore what happens when we add all the harmonics together. This can be done by first creating a row vector **p** containing 'ones', and then multiplying this with the matrix **S**:

```
p = ones(1,10);
f = p*S;
plot(f)
```

This is equivalent to calculating:

$$f_k(t) = \sum_{n=1}^{10} \sin(n\omega_s t_k) \qquad \textbf{(Eqn.1)}$$

Where ω_s = fundamental frequency and t_k = [0, tsamp, 2tsamp, ..., 127tsamp]. Explain the result **f**.

Instead of using a unity row vector, we could choose a different weighting **bn** for each harmonic component in the summation:

$$f_k(t) = \sum_{n=1}^{10} b_n \sin(n\omega_s t_k)$$

Try using **bn** = [1 0 1/3 0 1/5 0 1/7 0 1/9 0]. What do you get? What should **bn** be if we want to produce a saw-tooth waveform (use 10 terms only)? You may not be able to derive this during the lab session. Look it up in the library or work it out from first principles at home.

So far we have been using sine waves as our basis functions. Let us now try using cosine signals. Create a 10x128 matrix **C** contain 10 harmonics of cosine waveforms. Use the weighting vector **an** = [1 0 -1/3 0 1/5 0 -1/7 0 1/9 0], compute and examine **g=an*C**.

How does **g** differ from **f** obtained earlier from using sine waves? What general conclusions can you make about the sine and cosine series with relation to the even and odd nature of the signal?

The basis of the Fourier series is that a complex periodic waveform may be analyzed into a number of harmonically related sinusoidal waves, which constitute an orthogonal set. If we have a periodic signal $f(t)$ with a period equal to T, then $f(t)$ may be represented by the series:

$$f(t) = a_0 + \sum_{n=1}^{\infty} a_n \cos(n\omega_s t) + \sum_{n=1}^{\infty} b_n \sin(n\omega_s t) \qquad \textbf{(Eqn. 2)}$$

Prove that this equation can be rewritten as:

$$f(t) = a_0 + \sum_{n=1}^{\infty} c_n \cos(n\omega_s t - \phi_n) \qquad \textbf{(Eqn. 3)}$$

Derive the equations for c_n and ϕ_n in terms of a_n and b_n. What is the form of Eqn. 3 with **sine** as the orthogonal set?

From the above discussion, we have shown that almost any periodic signal can be expressed either as a set of sine and cosine waves of appropriate amplitude and frequency, or as a sum of sinusoidal components which are harmonically related and are defined by their *amplitudes* and *relative phases.*

A third form of the Fourier series, the exponential form, can be obtained by substituting the first of the following identities into Eqn. 3:

$$\cos(x) = \frac{1}{2}(e^{jx} + e^{-jx}) \qquad \text{and} \qquad \sin(x) = \frac{1}{2}(e^{jx} - e^{-jx})$$

Which gives:

$$f(t) = \sum_{n=-\infty}^{\infty} A_n e^{jn\omega_s t} \qquad \textbf{(Eqn. 4)}$$

Show the following:

- The coefficients of the exponential series are in general complex.
- The imaginary part of a coefficient A_n is equal but opposite in sign to that of coefficient A_{-n}.

In our discussion so far, we have deliberately omitted two important issues:

- How can the coefficients A_n be determined?
- What happens if the signal is not periodic?

These two questions will be dealt with in the lectures. It is sufficient for this experiment to state that the coefficients A_n are given by:

$$A_n = \frac{1}{T}\int_{-\frac{T}{2}}^{\frac{T}{2}} f(t)\, e^{-jn\omega_s t}\, dt \qquad \textbf{(Eqn. 5)}$$

We can further generalize Eqn. 4 and Eqn. 5 for a non-periodic signal by letting $T \to \infty$ and $\omega_s \to 0$. But we shall not consider this any further for this experiment.

Exercise 3 – DFT and Spectra of Signals

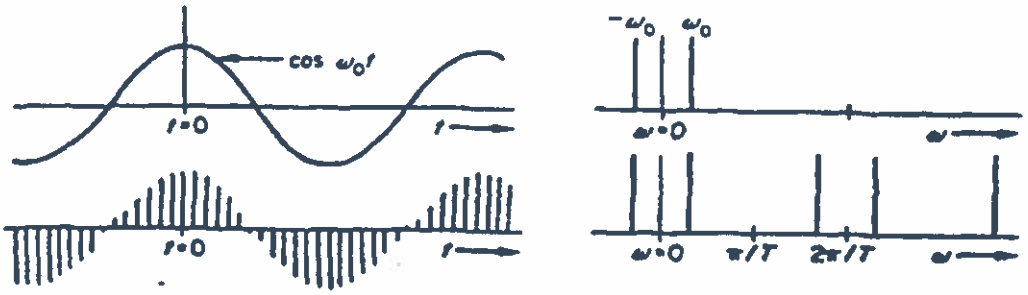


Figure 1 - A cosine signal and its spectrum

We have seen that a signal can be described either in the time-domain (as voltage samples) or in the frequency domain (as complex coefficients). The Discrete Fourier transform (DFT) defines the mapping between the discrete time domain and the discrete frequency domain of the sampled-data signals. The forward transform is from the time to the frequency domain, whereas the inverse transform is from the frequency to the time domain. Figure 1 shows a cosine signal in both continuous and sampled forms and their respective spectra. Note that the sampled-data spectra repeat indefinitely at intervals of $\omega = 2\pi/T$, where T is the sampling period. To calculate the spectrum of the periodic sample-data signal, we can rewrite the integral in Eqn. 5 above as a summation:

$$A_k(j\omega) = \frac{1}{N}\sum_{n=0}^{N-1} X_n e^{-j(2\pi kn)/N} \qquad \textbf{(Eqn. 6)}$$

Where X_n are the discrete time-domain samples, n is the sample number, k is the harmonic number and N is the total number of samples of the signal.

The number of multiplications needed for an N-point DFT is proportional to N^2. The algorithm is said to be of order $O(N^2)$. Fast algorithms have been developed to calculate the DFT (called Fast Fourier Transforms or FFT), which require only approximately $Nlog_2N$ multiplications.

Let us try the built-in FFT function to explore the spectra of various signals. The two functions `fft(x)` and `ifft(x)` implement the forward and inverse transforms respectively according to the following equations:

$$\text{Forward:} \quad A_{k+1} = \sum_{n=0}^{N-1} X_{n+1} e^{-j(2\pi kn)/N} \qquad \textbf{(Eqn. 7)}$$

$$\text{Inverse:} \quad X_{n+1} = \frac{1}{N} \sum_{k=0}^{N-1} A_{k+1} e^{-j(2\pi kn)/N} \qquad \textbf{(Eqn. 8)}$$

Where k is the harmonic or frequency bin number, n is the sample number and N is the number of samples in x. Note that the series is written in an unorthodox way, running over $n+1$ and $k+1$ instead of the usual n and k because vectors in **MATLAB** are indexed from 1 to N instead of from 0 to $N-1$. Note also that Eqn. 6 and Eqn. 7 differ by a factor of N. This is due to the fact that the transform can be defined mathematically in different ways. Eqn. 7 is a more popular formulation, but Eqn. 6 is easier to relate to the Fourier series.

Now let us find the spectrum of the sine wave produced with `sinegen`:

```
x = sinegen(8000,1000,8);
A = fft(x)
```

Explain the numerical result in `A`. Make sure that you know the significant of each number you get back. If in doubt, ask a demonstrator. You might also like to evaluate the FFT of a cosine signal, is it what you would expect?

Next, let us try to find the spectrum for an impulse with 16 samples:

```
x(1:16) = zeros(1,16);
x(1) = 1;
```

Examine the spectrum of `x` in terms of real and imaginary parts and magnitude and phase. Then, delay this impulse signal by 1 sample and find its spectrum again. Examine the real and imaginary spectra. Why are they different for the 2 signals? What do you expect to get if you delay the impulse by 2 samples instead of 1?

Instead of working with real and imaginary numbers, it is often more convenient to work with the magnitude and phase of the components, as defined by:

```
amp = sqrt ( A .* conj (A));
phase = atan2( imag(A), real(A));
```

If you precede the `*` or `/` operators with a period (`.`), **MATLAB** will perform an element-by-element multiplication or division of the two vectors instead of the normal matrix operation. `conj(A)` returns the conjugate of each elements in `A`. Investigate how the phase spectrum changes as you gradually increase the delay of the impulse; you might try the `unwrap` function.

Write a function `plotspec(x)` which plots the magnitude and phase spectra of `x` (the magnitude spectrum should be plotted as a bar graph). This function will be useful later.

Exercise 4 – The effects of using Windows

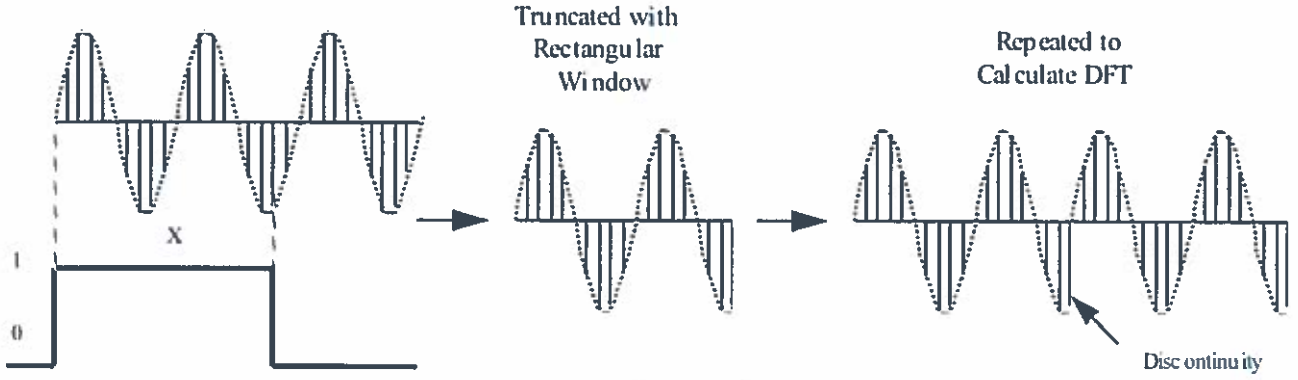


Figure 2 - Rectangular Window

So far we have only dealt with sine waves with an integer number of cycles. Generate a sine wave with an incomplete last cycle using:

```
x = sinegen(20000,1000,128);
```

Obtain its magnitude spectrum. Instead of obtaining just two spikes for the positive and negative frequencies, you will see many more components around the main signal component. Since we are not using an integer number of cycles, we introduce discontinuity after the last sample as shown in Figure 2. This is equivalent to multiplying a continuous sine wave with a rectangular window as shown in Figure 2. It is this discontinuity that generates the extra frequency components. To reduce this effect, we need to reduce or remove the discontinuities. Choosing a window function that gradually goes to zero at both ends can do this.

Two common window functions are defined mathematically as:

$$\text{Hanning Window:} \quad w[n] = 0.5\left(1 - \cos\left(2\pi\frac{n}{N-1}\right)\right)$$

$$\text{Blackman Window:} \quad w[n] = 0.42 - 0.5\cos\left(2\pi\frac{n-1}{N-1}\right) + 0.08\cos\left(4\pi\frac{n-1}{N-1}\right)$$

Where $n = 1, 2, \ldots, N$ for both windows.

MATLAB provides a number of window functions. Compare the magnitude spectrum of the sine wave without windowing to those weighted by the **Hanning** and **Blackman** window functions. Comment on the results.

Exercise 5 – The analysis of an unknown signal

To test how much you have understood so far, you are given an unknown signal stored in a disk file. The file contains over 1000 data samples. It is also known that the signal was sampled at 1 kHz and contains one or more significant sine waves. Take at least two 256-sample segments from this data and compare their spectra. Are they essentially the same? Make sure that you can interpret the frequency axis (ask a demonstrator if you are not sure). Find out the frequency and magnitude of its constituent sine waves. You can load the data into a vector using the **MATLAB** command:

```
load unknown;
```

2. What is a Digital Filter?

The purpose of a digital filter, as for any other filter, is to enhance the wanted aspects of a signal, while suppressing the unwanted aspects. In this experiment, we will restrict ourselves to filters that are linear and time-invariant. A filter is linear if its response to the sum of the two inputs is merely the sum of its responses of the two signals separately, while a time-invariant filter is one whose characteristics do not alter with time. We make these two restrictions because they enormously simplify the mathematics involved. Linear time-invariant filters may also be made from analogue components and it is often cheaper to do so. The advantages of digital filters lie in their repeatability with time, i.e. they do not suffer from ageing or temperature effects, their ability to handle very low frequencies and the ease with which their parameters may be altered. In addition, non-linear or time varying filters are very much easier to implement as digital filters than as analogue ones.

Exercise 6 – Filtering in the Frequency Domain

An obvious way to filter a signal is to remove the unwanted spectral components in the frequency domain. The steps are:

1. Take the FFT of the signal to convert it into its frequency domain equivalent.
2. Set unwanted components in the spectrum to zero.
3. Take the inverse FFT of the resulting spectrum.

Filter the noise-corrupted signal by removing all components above 300 Hz. Compare the filtered waveform with the original signal. What are the disadvantages of filtering in the frequency domain?

3. Impulse Response and Convolution

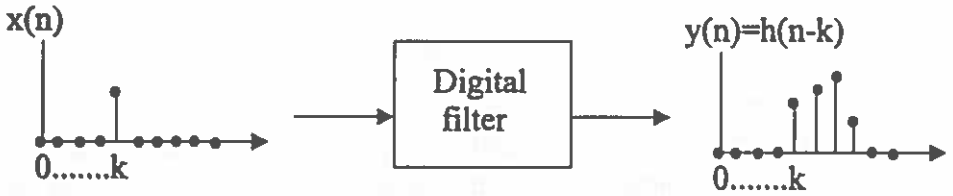


Figure 3 - Impulse response of a Filter

Let us represent the input sequence applied to a digital filter by **x(n)** and the corresponding output sequence by **y(n)**. Suppose we apply an impulse to the filter at a particular time, **n = k** as shown in Figure 3. The output sequence **y(n)** will be zero for **n < k** and may have some value for **n > k**. The sequence of numbers **h(n)** where **n = 0, 1, 2, ..., ∞** is called the impulse response of the system. In Figure 3 the impulse response is shifted to start at **k**, i.e. **h(n-k)**, due to the shift in the input. Once the impulse response is known, the output sequence of a filter can be evaluated for any input signal. We can illustrate this by regarding the input sequence as the sum of many impulses occurring at different sampling intervals, the height of the sample at position **k** is **x(k)**. Since the filter is linear, each impulse causes an output **h(n-k)*x(k)** starting at position **k**. Therefore the total output **y(n)** is just the sum of the contributions from all the impulse responses from **x(n)**, **x(n-1)**, **x(n-2)**, ... **x(0)**, as depicted in Figure 4. Thus:

$$y[n] = \sum_{k=0}^{n} h[n-k] * x[k] \qquad \textbf{(Eqn. 9)}$$

If we substitute r for n-k, this becomes:

$$y[n] = \sum_{r=0}^{n} h[r] * x[n-r] \qquad \textbf{(Eqn. 10)}$$

Thus for any linear time-invariant filter, the output values consist of a weighted sum of the past input values, with the weights being the elements of the impulse response. The operation described by Eqn. 10 is known as discrete convolution. Here, the input signal **x(n)** is said to be convolved with the impulse response of the filter **h(n)**.

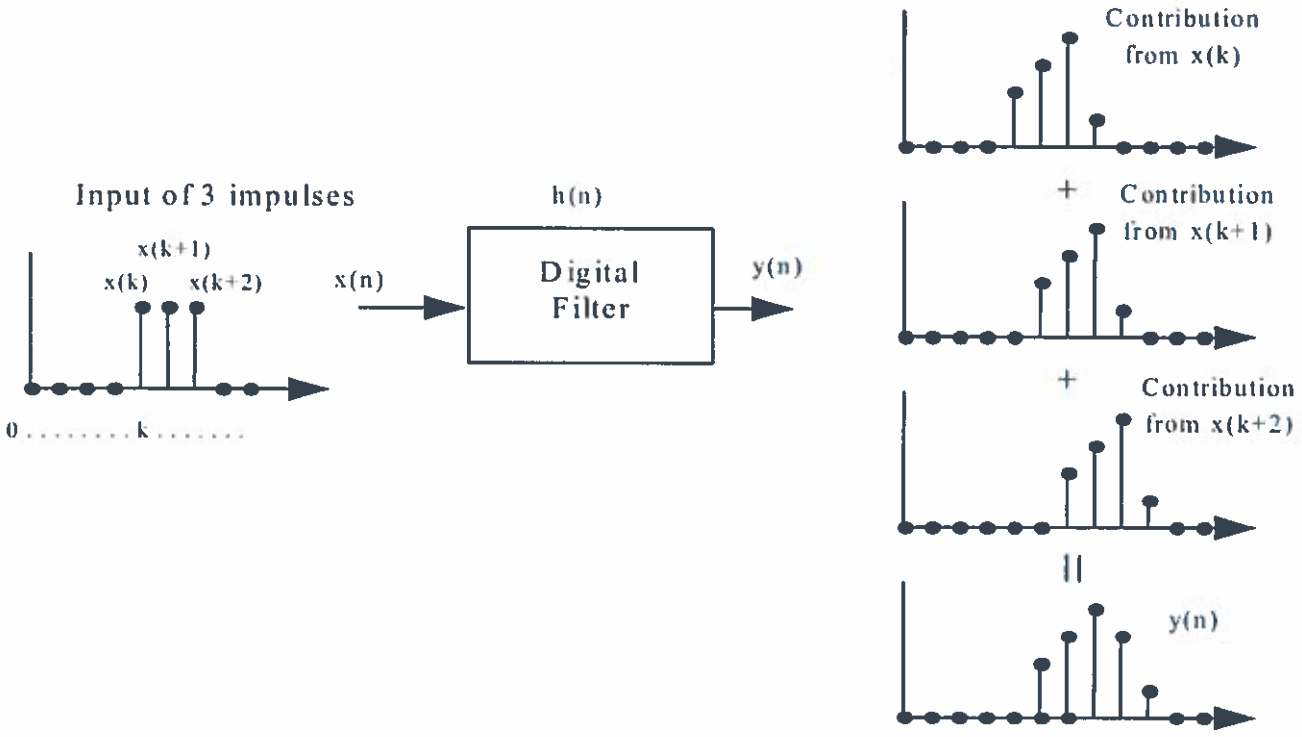


Figure 4 - Discrete Convolution

Exercise 7 – Spectra Pulse and Impulse

Create a pulse signal containing 8 samples of ones and 8 samples of zeros. Obtain its amplitude spectrum and check that it is as you expected. Gradually reduce the width of the pulse until it becomes an impulse, i.e. contains only a single one and 15 zeros. Note how the spectrum changes. Remember the effect of adding a larger and larger number of sine waves together in exercise 2?

4. Frequency Response of a Filter

In the previous exercise, we have seen that an impulse signal contains equal power in all frequency components, i.e. its magnitude spectrum is flat. Therefore the frequency response of a filter is simply the discrete Fourier transform of the filter's impulse response.

Exercise 8 – A simple Finite Impulse Response (FIR) Filter

We are now ready to try out a simple digital filter. Let us assume that the impulse response of the filter **h(n)** is a sequence of 4 ones. Find out the frequency response of this filter. This filter keeps the low frequency components intact and reduces the high frequency components. It is therefore known as a low-pass filter. Suppose the input signal to the filter is **x(n)**, show that this filter produces the output **y(n)** as:

$$y[n] = x[n] + x[n-1] + x[n-2] + x[n-3] \qquad \textbf{(Eqn. 11)}$$

Let us apply this filter to the noise-corrupted signal used in exercise 5. This can be done using the **conv(x)** function in **MATLAB** to perform a convolution between the signal and **h(n)**. Compare the waveform before and after filtering.

Another way of looking at this filter is that the output is produced by averaging 4 consecutive samples. This averaging action tends to smooth out fast varying frequency components, but has little effects on the low frequency components. Such a filter is often termed as a moving average.

Exercise 9 – Infinite Impulse Response (IIR) Filter

The filter described in the last exercise belongs to a class known as **Finite Impulse Response (FIR)** filters. It has the property that all output samples are dependent on input samples alone. For an impulse response that contains many non-zero terms, the amount of computation required to implement the filter may become prohibitively large if the output sequence (**y(n)**) is computed directly as a weighted sum of the past input values. Instead, **y(n)** is often computed as a sum of both past input values and past output values. That is:

$$y[n] = \sum_{r=0}^{nb} b[r] * x[n-r] + \sum_{r=1}^{na} a[r] * y[n-r] \qquad \textbf{(Eqn. 12)}$$

This type of filter has an impulse response of infinite length, therefore it is known as an **Infinite Impulse Response (IIR)** filter. It is also called a recursive filter because the outputs are being fed back to calculate the new output.

Derive the impulse response of the simple filter described by the following equation:

$$y[n] = 0.5x[n] + 0.5y[n-1] \qquad \textbf{(Eqn. 13)}$$

MATLAB provides a function called `filter` to perform general **FIR** and **IIR** filtering. It essentially implements Eqn. 12 with the syntax:

```
y = filter(b, a, x)
```

Where **b** = [b_0 b_1 ... b_n] and **a** = [1 a_1 a_2 ... a_n].

Filter the noise corrupted signal with an **IIR** filter having the following filter coefficients:

b = [0.0039 0.0193 0.0386 0.0386 0.0193 0.0039]
a = [1.0000 -2.3745 2.6360 -1.5664 0.4929 -0.0646]

Find the impulse response of this filter and hence its frequency response. Hence explain the effect of filtering the noise-corrupted signal with this filter.

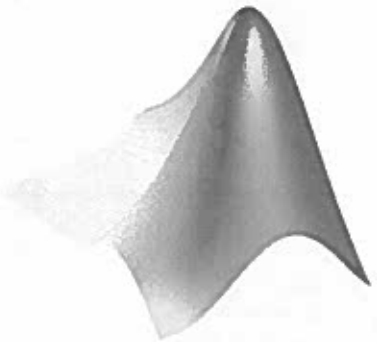
This filter (known as a **Butterworth Filter**) was designed using the **MATLAB** functions `buttord` and `butter`. If you have time, you might explore and design your own filters.

MAT Activity – Matlab Tutorial

2nd Year LAB EEE & ISE

IMPERIAL COLLEGE LONDON

Revised July 2011



Contents

Introduction 2
Vectors and Matrices 2
Defining a Vector 2
Defining Matrices 4
Matrix Functions 5
Matrix Operations 6
Plots 7
Examples 7
Flow Control 8
For Loops 8
While Loops 8
If Statements 8
Vectors *vs.* Loops 9
MATLAB Functions and Scripts 9
Scripts 9
Functions 10
Data Files 10
Saving and Recalling Data 10
Saving a Session as Text 11
Read/Write Non-.mat Files 11
Data Communications 12

Introduction

MATLAB is a programming language for calculations with vectors and matrices. The interface follows a language that is designed to look a lot like the notation use in linear algebra. In the following tutorial, we will discuss some of the basics of working with vectors.



After starting MATLAB it will wait for you to enter your commands in the **command window**. In the text that follows, any line that starts with two greater than signs (>>) is used to denote the MATLAB command line. This is where you enter your commands. On the left side you have a list of the files available in your working directory and on the right the list of variables currently in memory.

In the MATLAB command window you can assign values to alphanumeric variables as in a programming language. These values can be scalars (numbers), vectors, or 2X2 matrices. You do not have specify variable type, or even declare the variable. Any object can be assigned to any MATLAB variable, MATLAB is not statically typed, even though at any time a variable has a type (scalar, 3-vector, etc).

MATLAB syntax is **case sensitive**. This is another potential source of problems when you start building complicated algorithms.

```
>> A =
1
>> a
??? Undefined function or variable 'a'.
```

Matlab is shipped with a number of demonstration programs. Use help demos to find out more about these (the number of demos will depend upon the version of Matlab you have).

Some of the standard demos may be especially useful to users who are beginners in linear algebra:

demo - Demonstrate some of MATLAB's capabilities.

matdemo - Introduction to matrix computation in MATLAB.

rrefmovie - Computation of Reduced Row Echelon Form

If you type help MATLAB/general at command prompt it will give you the list of general functions, and description, available in MATLAB.

In the GUI help->product help->MATLAB gives you extensive help, e.g. the user guide

Vectors and Matrices

This is the basic introduction to MATLAB. Creation of vectors and matrices is included with a few basic operations. Topics include the following:

- Defining a vector
- Accessing elements within a vector
- Basic operations on vectors
- Defining Matrices
- Matrix Functions
- Matrix Operations

Defining a Vector

Almost all of MATLAB's basic commands revolve around the use of vectors. A vector is defined by placing a sequence of numbers within square braces:

```
>> x = [2 5]
x =
2 5
```

This creates a row vector which has the label *x*. The first entry in the vector is a 2 and the second entry is a 5. Note that MATLAB printed out a copy of the vector after you hit the enter key. If you do not want to print out the result put a semi-colon at the end of the line:

```
>> x = [2 5];
>>
```

If you want to view the vector just type its label:

```
>> x
x =
2 5
```

You can define a vector of any size in this manner:

```
>> fib = [1 2 3 5 8 13]
fib =
1 2 3 5 8 13
```

Notice, though, that this always creates a row vector. If you want to create a column vector you need to take the

transpose of a row vector. The transpose is defined using an apostrophe ('):

```
>> fib = [1 2 3 5 8 13]'
fib =
1
2
3
5
8
13
```

A common task is to create a large vector with numbers that fit a repetitive pattern. MATLAB can define a set of numbers with a common increment using colons. For example, to define a vector whose first entry is 1, the second entry is 2, the third is three, up to 7 you enter the following:

```
>> x = [1:7]
x =
1 2 3 4 5 6 7
```

If you wish to use an increment other than 1, then you have to define the start number, the value of the increment, and the last number. For example, to define a vector that starts with 2 and ends in 4 with steps of .25 you enter the following:

```
>> x = [2:.25:4]
x =
2.0000 2.2500 2.5000 2.7500 3.0000 3.2500
3.5000 3.7500 4.0000
```

You can view individual entries in this vector. For example to view the first entry just type in the following:

```
>> x(1)
ans =
2
```

This command prints out entry 1 in the vector. Note that all vectors in MATLAB are indexed starting with 1 going upwards. This is different from C, in which all arrays start at 0. Also notice that a new variable called *ans* has been created. Any time you perform an action that does not include an assignment MATLAB will put the label *ans* on the result. To simplify the creation of large vectors, you can define a vector by specifying the first entry, an increment, and the last entry. MATLAB will automatically figure out how many entries you need and their values. For example, to create a vector whose entries are 0, 2, 4, 6, 8 and 10, you can type in the following line:

```
>> 0:2:10
ans =
0 2 4 6 8 10
```

MATLAB also keeps track of the last result. In the previous example, a variable *ans* is created. To look at the transpose of the previous result, enter the following:

```
>> ans'
ans =
0
2
4
6
8
10
```

To be able to keep track of the vectors you create, you can give them names.
For example, a row vector x can be created:

```
>> x = [0:2:10]
x =
0 2 4 6 8 10
>> x
x =
0 2 4 6 8 10
>> x;
>> x'
ans =
0
2
4
6
8
10
```

Note that in the previous example, if you end the line with a semi-colon, the result is not displayed. This will come in handy later when you want to use MATLAB to work with very large systems of equations.
MATLAB has a special operation called vector slicing which generates a sub-vector from parts of a vector. If you want to only look at the first three entries in a vector you can use the same notation you used to create the vector:

```
>> x(1:3)
ans =
0 2 4
>> x(1:2:4)
ans =
0 4
>> x(1:2:4)'
ans =
0
4
```

Once you understand the notation you are free to perform other operations:

```
>> x(1:3)-x(2:4)
ans =
-2 -2 -2
```

For the most part MATLAB follows the standard notation used in linear algebra. We will see later that there are some extensions to make some operations easier. For now, though, both addition and subtraction are defined in the standard way. For example, to define a new vector with the numbers from 0 to -5, in steps of -1, we do the following:

```
>> y = [0:-1:-5]
y = [0:-1:-5]
y =
0 -1 -2 -3 -4 -5
```

We can now add x and y together in the standard way:

```
>> x + y
ans =
0 1 2 3 4 5
```

Additionally, scalar multiplication is defined in the standard way. Also note that scalar division is defined in a way that is consistent with scalar multiplication:

```
>> -2*y
ans =
0 2 4 6 8 10
>> x/3
ans =
0 0.6667 1.3333 2.0000 2.6667 3.3333
```

With these definitions linear combinations of vectors can be easily defined and the basic operations combined:

```
>> 2 * x - y
ans =
0 5 10 15 20 25
```

These operations can only be carried out when the dimensions of the vectors allow it. You often see the following error message which follows from adding two vectors whose dimensions are different:

```
>> 2 * x - [1 3]
??? Error using ==> minus
Matrix dimensions must agree.
```

Defining Matrices

Defining a matrix is similar to defining a vector. To define a matrix, you can treat it like a column of row vectors (note that the spaces are required!):

```
>> A = [ 1 2 3; 3 4 5; 6 7 8]
A =
1 2 3
3 4 5
6 7 8
```

You can also treat it like a row of column vectors:

```
>> B = [[1 4 7]' [2 5 8]' [3 6 9]']
B =
1 2 3
4 5 6
7 8 9
```

(It is important to include the spaces.)
If you have been putting in variables through this tutorial, then you probably have a lot of variables defined. If you lose track of what variables you have defined, the *whos* command will let you know all of the variables you have in your workspace.

```
>> whos
Name Size Bytes Class
A 3x3 72 double array
B 3x3 72 double array
x 1x6 40 double array
y 1x6 40 double array
```

As mentioned before, the notation used by MATLAB is the standard linear algebra notation you should have seen before. Matrix-vector multiplication can be easily done. You have to be careful, though, your matrices and vectors have to have the right size!

```
>> x = [1:3:9]
x =
1 4 7
>> A*x
??? Error using ==> *
Inner matrix dimensions must agree.
>> A*x'
ans =
30
54
90
```

Note the difference between a row vector (x) and a column vector (x'). Row vectors only pre-multiply matrices, column vectors only post-multiply matrices. You will see that particular error message whenever you start using matrices and vectors, forgetting about their sizes.

You can work with different parts of a matrix, just as you can with vectors.

When slicing an array use a comma to separate the row indexes from the column indexes. The row indexes (specifying which rows) come first. Again, you have to be careful to make sure that the operation is legal.

```
>> A(1:2,3:4)
??? Index exceeds matrix dimensions.
>> A(1:2,2:3)
ans =
2 3
4 5
>> A(1:2,2:3)'
ans =
2 4
3 5
```

You can also access rows and columns within a matrix.

```
>> A(1,:)
ans =
1 2 3
>> A(:,1)
ans =
1
3
6
```

Matrix Functions

You can perform many standard operations on matrices. For example, you can find the main diagonal of a matrix.

```
>> diag(A)
ans =
1
4
8
>>
```

Other operations include finding the determinant of a random square matrix with size 3. A diagonal of a matrix is extracted into a vector with command diag.

```
>> A = rand(3,3)
A =
0.9649 0.9572 0.1419
0.1576 0.4854 0.4218
0.9706 0.8003 0.9157
>> det(A)
ans =
0.3079
>> diag(A)
ans =
0.9649
0.4854
0.9157
```

Summary

>> MATLAB command prompt

MATLAB is a case-sensitive, **dynamically typed**, interpreted language. Types are: **scalar, 2D matrix**. Matrices have a precise size. Scalars are real (double floating point) or complex numbers and all matrix elements are scalars. **Vectors** are special case of matrix: size (1XN) is row vector, size (NX1) is column vector.

Arithmetic operators applied to matrices and vectors perform standard linear algebra. Sizes must match. Transpose operator ' can be used to turn row into column vectors etc.

Matrices **M** (and vectors **v**) can be **sliced** to get sub-matrices or **selected** to get elements. Matrices are always numbered upwards from 1.

M(1:2,:) first two rows of M
M(:,2:3) columns 2-3 of **M**
M(1:2,1:2) top left 2X2 submatrix of **M**
M(1,2) element in row 1 col 2 of M
M(:,2) column vector from col 2 of M
v(1:4) first 4 elements of v
v' transpose of **v** (row -> col or col -> row)
v * v' scalar product v^2
v(3) third element of v

It is good practice but not required to use upper-case for matrix names, lower-case for vector names.

Math scalar functions apply to vectors as well as scalars, and result in vectors using elementwise application:
sin(v) is a vector of the sin of elements in **v**
whos is useful command tells you current **types** of all defined variables
help is invaluable command to tell you more about any MATLAB command from command line
From GUI help->function browser can be used to search or browse very large number of built-in functions
From GUI help->product documentation->MATLAB-> gives excellent general documentation

Matrix Operations

There are also routines that let you find solutions to equations. For example, if Ax=b and you want to find *x*, a slow way to find *x* is to simply invert *A* and perform a left multiply on both sides (more on that later). It turns out that there are more efficient and more stable methods to do this (LU decomposition with pivoting, for example). MATLAB has special commands that will do this for you.

Before finding the approximations to linear systems, it is important to remember that if A and B are both matrices, then AB is not necessarily equal to BA. To distinguish the difference between solving systems that have a right or left multiply, MATLAB uses two different operators, \ and /. Examples of their use are given below. It is left as an exercise for you to figure out which one is doing what.

```
>> v = [1 3 5]'

v =
1
3
5
>> B = [[1 3 5]' [2 4 8]' [4 6 8]'];
>> x = B\v

Warning: Matrix is close to singular or
badly scaled.
Results may be inaccurate. RCOND =
1.541976e-018.
```

```
x =
1.0000
0.0000
-0.0000
>> B*x
ans =
1
3
5
>> x1 = v'/B
x1 =
1.8333 -0.8333 1.3333
>> x1*B
ans =
1.0000 3.0000 5.0000
```

Finally, sometimes you would like to clear all of your data and start over. You do this with the clear command. Be careful though, it does not ask you for a second opinion and its results are final.

```
>> clear
>> whos
```

Plots

If you issue the command help plot, MATLAB will give details on the usage of the command to produce plots from existant data.

```
>> help plot
PLOT Linear plot.
PLOT(X,Y) plots vector Y versus vector X.
(...)
```

Like many MATLAB functions the behavior of plot depends on the type of object you give it. Vectors are taken as sequences of y values. Matrices are taken as multiple sequences of y values. Complex vectors are taken as sequences of points on an Argand diagram. Below there is a simple example to show how the plot command is used.

```
>> x = 0:2*pi/100:2*pi;
>> y = sin(x);
>> z = cos(x);
>> size(y)
ans =
1 101
>> max(size(y))
ans =
101
>> y(1)
ans = 0
>> plot(x,y)
Add grid to the plot
>> grid on
```

Now, we have two sets of data for the two functions used (sin, cos). To compare the two in one plot, we give it the range, the domain, and the format.

To do this, you have to tell MATLAB that you want two plots in the picture. This is done with the subplot command. Subplot command takes 3 arguments (m,n,p) and it breaks the figure into a m-by-n matrix of plots while p selects the current plot, starting at top-left position.

Here we will have one row and two columns giving us two plots. In plot 1 the *sin* function is plotted, while in plot 2 the *cos* function is plotted.

```
>> subplot(1,2,1);
>> plot(x,y)
>> grid on;
>> subplot(1,2,2);
>> plot(x,z)
>> grid on;
```

If you want to save the plot to a file. You can use the following example to create a postscript file called plot.ps which resides in the current directory.

```
>> print -dps plot.ps
```

Examples

The following examples can be found in MATLAB help for each command.

```
>> [c,h] = contour(peaks); clabel(c,h),
colorbar
```

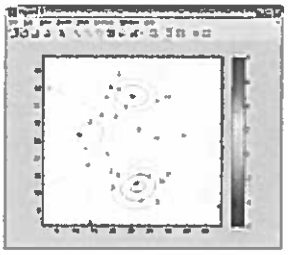


Figure 1: Contour demo.

```
>> x = -4:0.1:4; y = randn(10000,1);
hist(y,x)
>> [X,Y] = meshgrid(-2:.2:2, -2:.2:2);
Z = X .* exp(-X.^2 - Y.^2);
surf(X,Y,Z)
```

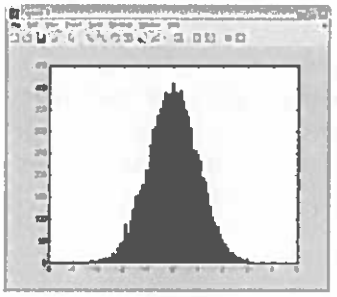


Figure 2: Histogram demo.

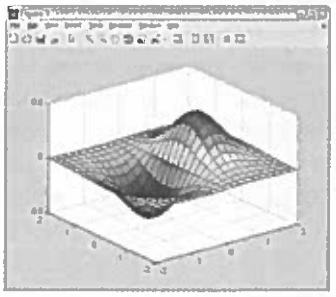


Figure 3: Surface demo.

Summary

x = B \ v (solves Bx = v)
x = B / v (solves xB = v)
plot(x,y) plots coordinates (x(i),y(i))
subplot(xsize,ysize,subfig) plots subgraph subfig in matrix of subgraphs (xsize,ysize).
hist(vals,bins) plots histogram of values using vector bins to separate bars on x axis

Flow Control

For Loops

MATLAB for loops iterate a loop variable over the elements of a vector. In general any number of statements are allowed in the loop body between **for** and **end**.

```
>> for j=1:4,
j
end
j =
1
j =
2
j =
3
j =
4
>>
```

Here is another example:

```
>> v = [1:3:10]
v =
1 4 7 10
>> for j=1:3,
v(j) = 3*j;
end
>> v
v =
3 6 9 10
```

Note this is an unrealistic example. The notation used in the first statement is much faster and easier to write than the loop.

A better example is one in which we want to perform operations on the rows of a matrix. If you want to start at the second row of a matrix and subtract the previous row of the matrix and then repeat this operation on the following rows, a **for** loop can do this easily:

```
>> A = [ [1 2 3]' [3 2 1]' [2 1 3]']
A =
1 3 2
2 2 1
3 1 3
>> B = A;
>> for j=2:3,
A(j,:) = A(j,:) - A(j-1,:)
end
A =
1 3 2
1 -1 -1
3 1 3
A =
1 3 2
1 -1 -1
2 2 4
```

For a more realistic example, since we can now use loops and perform row operations on a matrix, Gaussian Elimination can be performed using only two loops and one statement> Note how for loops are nested here.

```
>> for j=2:3,
for i=j:3,
B(i,:) = B(i,:) - B(j-1,:)*B(i,j-1)/B(j-
1,j-1)
end
end
B =
1 3 2
0 -4 -3
3 1 3
B =
1 3 2
0 -4 -3
0 -8 -3
B =
1 3 2
0 -4 -3
0 0 3
```

While Loops

The **while** loop is often used in MATLAB programs, it terminates when the specified condition is no longer met:

```
>> while ((a*a') < 1)
a = 2 * a
end
```

If Statements

MATLAB **if** statements have the usual forms with optional **elseif** and/or **else** parts:

if condition statements **end**	**if** condition statements **else** statements **end**	**if** condition statements **elseif** condition statements **end**

MATLAB allows you to combine multiple Boolean expressions using the logic operators, & (and), | (or), and _ (not). Note the difference from C. For example to check to see if *a* is less than *b* and at the same time *b* is smaller than *c* you would use the following commands:

```
>> if (a < b) & (b < c)
disp('a is smaller than c!')
end
```

Note that **if** statements can be written on a single line using **;** as separator:

```
>> if (a < b); tmp = b; b = a; a = tmp end
```

Summary

```
for j = [1:10]          while i < 10        if (a < b)          if c == 1           if (a > 0) & (a < 1)
  x(j) = x(j)*2           a = a * 2           tmp = a             ....                b = a
end                       i = i - 1           a = b             elseif  c == 2      else
                        end                   b = tmp             ....                b = -a
                                            end                 end                 end
```

Vectors *vs.* Loops

Matlab is optimized for vectors. Most of its basic functions are optimized to compute vectors instead of scalars. Therefore **you should always vectorize your code.**

To demonstrate this statement, please try the code below and verify the times taken by the different implementations of the same algorithm.

```
N = 1000;

A = rand(N);

tic()               % start time counter
d1 = diff(A);
t1 = toc()          % stop timer counter

tic()
for i = 1:N-1
    d2(i,:) = A(i+1,:) - A(i,:);
end
t2 = toc()
fprintf(1,'delay: %f%%\n', 100*t2/t1);

tic()
for i = 1:N-1
    for j = 1:N
        d3(i,j) = A(i+1,j) - A(i,j);
    end
end
t3 = toc()
fprintf(1,'Delay: %f%%\n', 100*t3/t1);
```

MATLAB Functions and Scripts

MATLAB 'programs' are written in text files called an M-file with the extension ".m".

M-files can be either **scripts** or **functions**. Scripts are files containing a sequence of MATLAB statements, while functions make use of their own local variables, accept input arguments and can return values.

Scripts

In this example a file called *myscript.m* is created in the current directory. To get MATLAB to execute the commands in the file simply type in "myscript". MATLAB will then search the current directory for the file "myscript", read the file, and execute the commands in the file. If MATLAB cannot find the file you will get an error message:

??? Undefined function or variable 'myscript'.

If this is the case then either you mistyped the name of the program, the program is misnamed, or the file is located in a directory that MATLAB does not know about. In the later case you have to let MATLAB know which directory to search. The list of directories that is searched for files is called the path.

Invoke the MATLAB editor:

```
>> edit myscript.m
```

Either type or cut and paste the necessary MATLAB commands:

```
% file: myscript.m
% This MATLAB file creates a sin wave plot
% with 100 points
% Note that to generate 100 points there
% are 99 intervals between values
x = [0 : 2 * pi / 99 : 2 * pi];
y = offset + sin(x);
plot(x, y);
grid on;
axis tight;
title('Sine Wave')
```

Once the commands are in place, save the file. Go back to your original window and start up MATLAB. The file is called up by simply typing in the base name *myscript*.

```
>> myscript
??? Undefined function or variable
'offset'.
Error in ==> myscript at 12
y = offset + sin(x);
```

If you try to call the file without first defining the variable **offset** you will get an error message. You must first specify all of the variables that are not defined in the file itself.

```
>> offset = 1;
>> myscript
>> whos
Name Size Bytes Class Attributes
offset 1x1 8 double
x 1x100 800 double
y 1x100 800 double
There are 201 elements using 1608 bytes
>> plot(x,y,'rx',x,true)
```

When type in the command *myscript*, MATLAB reads the file and executes the commands as if you had typed them from the keyboard.
If you would like to run the program again with a different function for y, eg. cos(x), you have to be careful. The program will write over the vectors x and y. If you want to save these vectors, you must do so explicitly!

```
>> x0 = x+1;
>> y0 = y+1;
>> myscript
>> whos
Name Size Bytes Class Attributes
offset 1x1 8 double
x 1x100 800 double
y 1x100 800 double
x0 1x100 800 double
y0 1x100 800 double
>> plot(x0,y0,'gx',x,y,'yo');
```

Now you can compare different results from your script in the same plot.

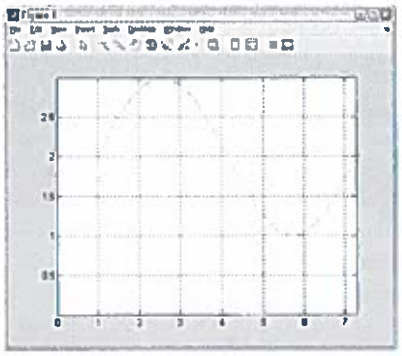


Figure 4: Script demo.

MATLAB does not require the correct indentation but it is strongly recommended you indent blocks in scripts or functions as this makes your code MUCH easier to read.

Functions

New functions can be added to MATLAB by expressing them in terms of existing functions.
If you are not familiar with a more advanced editor use MATLAB's built in editor to create the file. Type in the following command at the MATLAB prompt:

>> edit myscript.m

The line at the top of a function M-file contains the syntax definition. The name of a function, as defined in the first line of the M-file, should be the same as the name of the file without the .m extension.
Sub-functions can also be defined within a function, but they will only be visible for function in the file where they are defined.

```
function [var2,var3] = myfunc(vec_x)
    var1 = length(vec_x);
    var2 = mysubfunc(vec_x, var1);
    if var2 > var1
        var3 = var2 * var1;
    else
        var3 = var2 / var1;
    end

function var2 = mysubfunc(x, var1);
    var2 = sum(x) / var1;
```

After you created the file with the above code, you can try calling your function from MATLAB command line using the following alternatives:

1.
```
myfunc([-1 1 2 3])
```
2.
```
r = myfunc([-1 1 2 3])
```
3.
```
[r1, r2] = myfunc([-1 1 2 3])
```

What's the difference between them?

Data Files

MATLAB does allow you a number of different ways to save the data. In this tutorial we explore the different ways that you can save and read data into a MATLAB session.

Saving and Recalling Data
Saving a Session as Text
Read/Write Non-.mat Files

Saving and Recalling Data

As you work through a session you generate vectors and matrices. The next question is how do you save your work? Here we focus on how to save and recall data from a MATLAB session. The command to save all of the data in a session is save. The command to bring the data set in a data file back into a session is load.
We first look at the save command. In the example below we use the most basic form which will save all of the data present in a session. Here we save all of the data in a file called vars.mat. (.mat is the default extension for MATLAB data.)

```
>> x = [1 2 3];
>> y = [6 5 4];
>> whos
Name Size Bytes Class
x 1x3 24 double array
y 1x3 24 double array
Grand total is 6 elements using 48 bytes
>> save vars.mat
>> dir
.
..
vars.mat
```

The dir command is used to list all of the files in the current directory. In this situation we created a file called vars.mat which contains the vectors x and y. The data can be read back in to a MATLAB session with the load command.

```
>> clear
>> whos
>> load vars.mat
>> whos
Name Size Bytes Class
x 1x3 24 double array
y 1x3 24 double array
Grand total is 6 elements using 48 bytes
>> x+y
ans =
7 7 7
```

In this example the current data space is cleared of all variables. The contents of the entire data file, "vars.mat", is then read back into memory. You do not have to load all of the contents of the file into memory. After you specify the file name you then list the variables that you want to load separated by spaces. In the following example only the variable *x* will be loaded into memory.

```
>> clear
>> whos
>> load vars.mat x
>> whos
Name Size Bytes Class
x 1x3 24 double array
Grand total is 3 elements using 24 bytes
>> x
x =
1 2 3
```

Note that the save command works in exactly the same way. If you only want
to save a couple of variables you list the variables you want to save after the
file name. Again, the variables must be separated by a space. For an example
and more details please see the help file for save. When in MATLAB just
type in help save to see more information. You will find that there are large
number of options in terms of how the data can be saved and the format of the data file.
The delete command is used to remove the file created previously.

```
>> delete vars.mat
>> dir
.
..
```

Saving a Session as Text

MATLAB allows you to save the data generated in a session, but you cannot easily save the commands so that they can be used in an executable file. You can save a copy of what happened in a session using the diary command. This is very useful if you want to save a session for a homework assignment or as a way to take notes.
A diary of a session is initiated with the diary command followed by the file name that you want to keep the text file. You then type in all of the necessary commands. When you are done enter the diary command alone, and it will write all of the output to the file and close the file. In the example below a file called save.txt is created that will contain a copy of the session.

```
>> diary save.txt
... enter commands here...
>> diary
```

This will create a file called save.txt which will hold an exact copy of the output from your session.

Read/Write Non-.mat Files

In addition to the high level read/write commands detailed above, MATLAB allows C style file access. We do not go into great detail here. Instead we look at the basic commands. After looking at this overview we recommend that you look through the relevant help files.
This will help fill in the missing blanks. The basic idea is that you open a file, execute the relevant reads and writes on a file, and then close a file. One other common task is to move the file pointer to point to a particular place in the file, and there are two commands, *fseek* and *ftell* to help.
In the example below, a file called *temp.log* is opened. The file identifier is kept track of using a variable called fp. Once the file is opened the file position is moved to a particular place in the file (bof stands for "begin of file"), denoted pos, and two double precision numbers are read. Once that is done the position within the file is stored, and the file is closed.
Before running the code, make sure you create a text file named *temp.log* with the following content:

```
1,10
5,11
10,12
15,14
20,15
25,17
30,19
35,21
```

```
fp = fopen('temp.log','r');
tmp = fscanf(fp,'%d,%d',10);% read 10 items
% tmp = fscanf(fp, '%d,%d') %read all items
fclose(fp);
```

Matlab code to read the first 5 records from the file *temp.log* into *tmp* variable.

Summary - M-file content

```
%function definition - NB this comment is not needed
function [var2,var3] = myfunc(vec_x) %first line (other than comments) defines parameters and return values
% this function returns vector [var2,var3] and is called as myfunc(vec,x), i.e. two parameters.
   var1 = length(vec_x);
   var2 = mysubfunc(vec_x, var1); % this line defines first element of returned vector
   if var2 > var1
      var3 = var2 * var1; % defines second element of returned vector
   else
      var3 = var2 / var1; % alternate defn of second element
   end

function var2 = mysubfunc(x, var1); % first function is all of file unless another function is defined as here
   var2 = sum(x) / var1;
% note anything after % is comment and ignored
%script definition is like function but no header line
```

Data Communications

MATLAB has a very extensive library of functions. For example, it allows access to PC serial COM ports. This interface can be used to control/acquire data from an external peripheral, e.g. multimeter, digital thermometer, water pump, fan, etc. Below is a sample MATLAB program (script, in M-file) to query a measurement device attached to serial port COM1. Note the use of "exception" statement - if the serial port initiatlsation goes wrong, e.g. if port specified does not exist, the catch part will be executed to handle the error instead of the script terminating with error. In this case there is no way to continue after error so break will terminate the script after the error message has been printed.

```
COMM_PORT = 'COM1'
try
   % config serial port to use
   serial_port = serial(COMM_PORT, 'BaudRate', 19200, 'Timeout', 2000)
   fopen(serial_port);
catch exception
   disp('WARNING: Failed to open COM port!')
   break;
end

% To query the device.
fprintf(serial_port , '*IDN?');
idn = fscanf(serial_port);

% To disconnect the serial port object from the serial port.
fclose(s1);

disp(idn)
```